

L8: Order book and high frequency data modeling

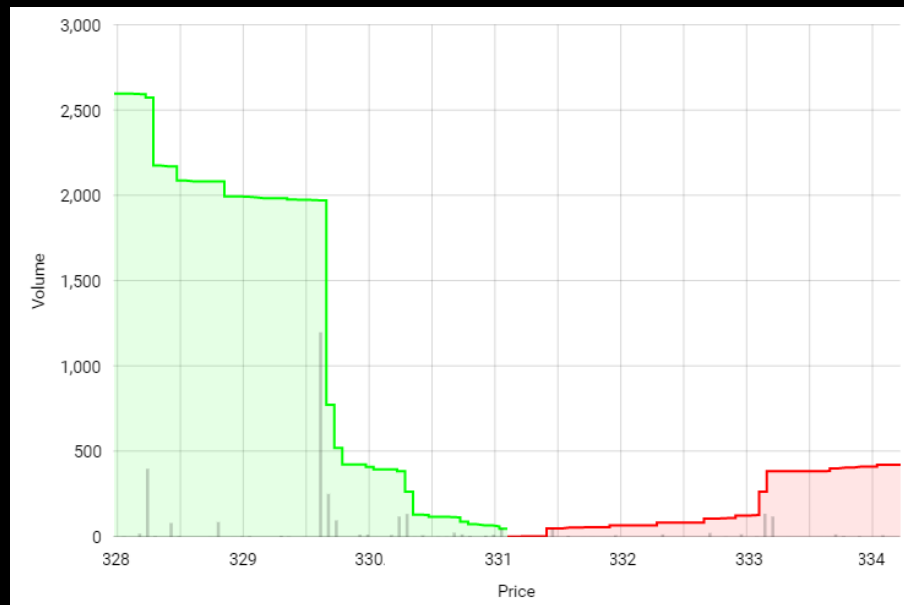
Course Code: COMP7415

Agenda

- Trading ideas and insights from an order book
- Market impact of a large order
- Queuing theory for order book modelling
- Introduction to high frequency trading
- Major types of arbitrage
- Triangular Arbitrage Strategy
- Market Making Strategy

Revision of Order Book

- Order book is a list of buy and sell orders
 - Bid book represents the demand side
 - Ask book represents the supply side
- An order book can provide insights for microeconomic structure

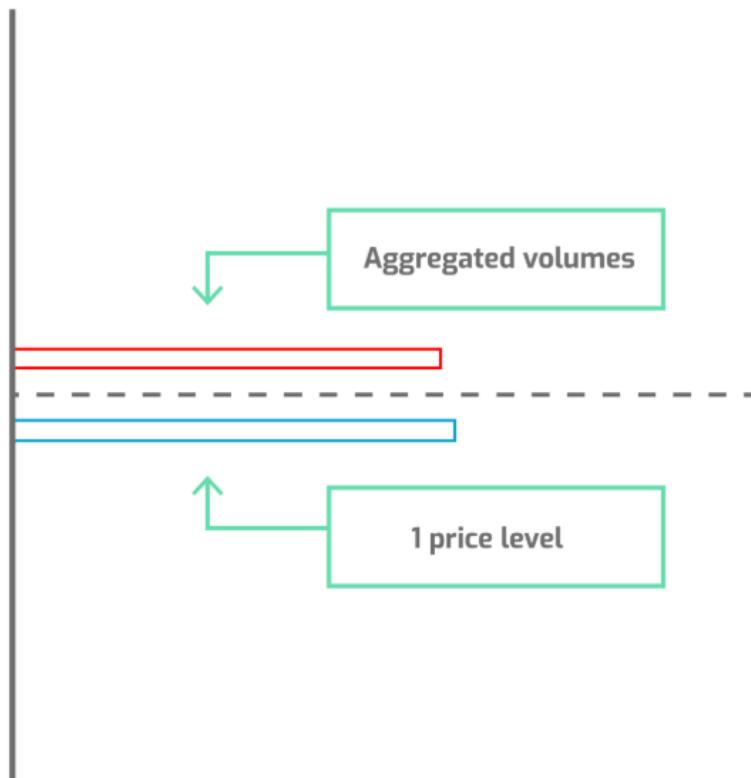


Order Book

	Bid	Ask	
3,151	9139.60	9152.80	167
1,574	9138.40	9153.21	1,600
338	9138.20	9153.60	736
789	9138.10	9153.71	1,602
800	9138.00	9154.10	1,649
309	9137.79	9154.21	1,965
2,140	9137.60	9154.50	3,984
48	9137.40	9154.60	763
144	9137.29	9154.71	157
136	9137.20	9154.80	56
528	9137.00	9155.50	320
1,654	9136.90	9155.71	200
40	9136.79	9156.10	16
46	9136.70	9156.21	219
608	9136.60	9156.30	40
48	9136.50	9156.40	80

Breakdown an Order Book

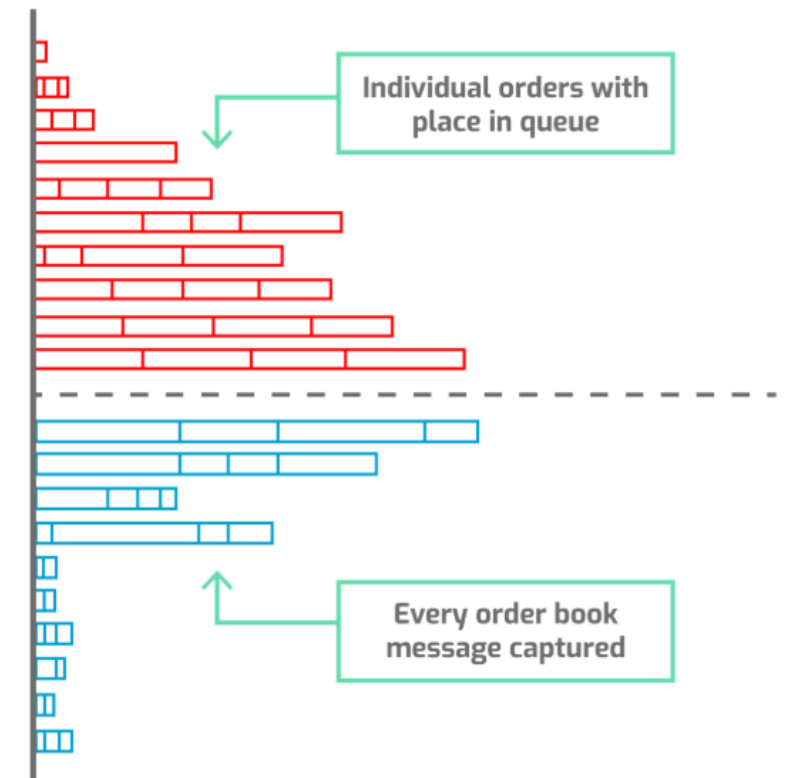
Level 1



Level 2

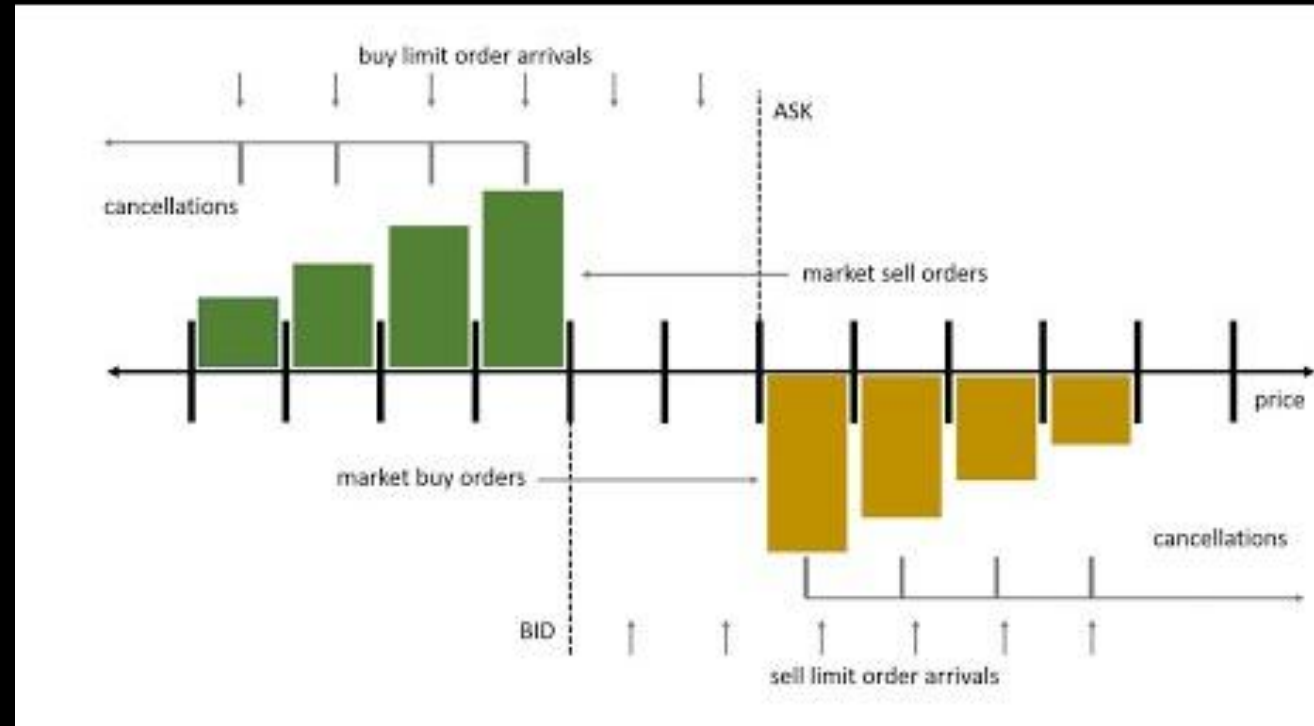


Level 3



Order Book Mechanism

- Limit orders operated under a price-time priority rule.
 - Limit orders are sorted by the price and higher priority is given to the orders at the best prices
 - Orders at the same price are ranked depending on when they entered the queue according to FIFO



What an order book can tell you?

1. **Supply & Demand:** Insights into trader behaviour and market sentiment.
2. **Support & Resistance Level:** Insights from orders with large volume
3. **Market Liquidity:** Measure liquidity through order sizes.
4. **Market Depth:** Analysis of the number of buy/sell orders at various price levels.

Trading Strategies Based on Order Book Data

1. **Momentum Trading:**

- Identifying trends through order flow signals.
- Monitor the ratio of buy to sell orders. A significant imbalance can signal potential price movements.

2. **Sniping Strategy:**

- Taking advantage of large orders (eg. iceberg orders) before they get fully executed.

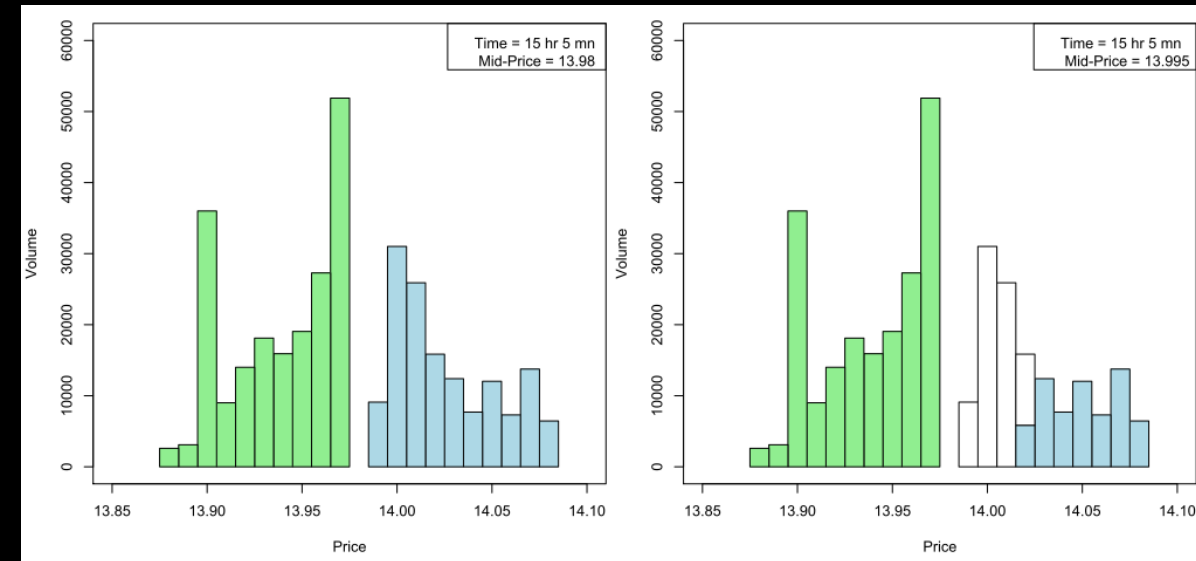
3. **Liquidity Provisioning:**

- Market-making strategies to earn from the bid-ask spread.

Market Impact

Market Impact

- A large market order is likely to fill up the order book and lead to
 - significant price movement
 - reduced market depth and liquidity
- Immediate Impact: Price change upon execution of the order.
- Temporary Impact: Short-term fluctuations that revert after order execution.
- Permanent Impact: Lasting price changes due to order flow



Factors Influencing Market Impact

1. **Order Size:**

- Larger orders typically lead to greater impact

2. **Order Type:**

- Market orders vs limit orders

3. **Market Conditions:**

- Less liquid markets experience higher impact
- Volatility and market volume affect impact

Strategies to Mitigate Market Impact

1. **Order Splitting:**

- Break large orders into smaller segments.
- Execute over time to reduce immediate impact.

2. **Use of Limit Orders:**

- Place limit orders to control execution price.
- Avoid the risks of market orders.

3. **Time-weighted Average Price (TWAP):**

- Execute orders evenly over a specified time period.
- Helps in minimizing market impact.

4. **Volume-weighted Average Price (VWAP):**

- Execute orders in proportion to market volume.
- Reduces price impact by blending into the market.

1. Order Splitting

- If informed investors' trades are the main cause of stock price changes, large trades by informed traders would reveal their information because traders can easily discern those who execute large transactions.
- However, small trades executed repeatedly will incur excessive transactions costs.
- Thus, informed traders will tend to fragment orders into medium-sized trades sufficiently small to disguise themselves but large enough to maintain fairly small transactions costs.

Example: Slicing the Large Order

- Suppose a client of the XYZ brokerage firm wants to buy $X = 80,000$ shares with the best possible execution.
- These execution costs B will include
 - order processing costs F and
 - market impact (slippage) costs that are increasing in the size c of each of n equal-size transactions.
- Broker XYZ works with a market impact model that estimates the slippage cost S of each execution of size $c = X/n$ with function $S = vc^m$, where the coefficient v and exponent m are related to the liquidity of the security:

$$B/n = F + v(X/n)^m$$

Example: Slicing the Large Order

- Suppose that broker estimated F to be \$0.03 per order, and v and m to be 0.00001 and 1.5. What are the optimal number of transactions and transaction sizes? We set up a total transactions cost function as follows:

$$B = nF + nv \left(\frac{X}{n} \right)^m = 0.03n + 0.00001 \frac{80,000^{1.5}}{n^{0.5}} = 0.03n + 226.27n^{-0.5}$$

- Take derivative of B with respect to n , and solve for n as follows:

$$\frac{dB}{dn} = \frac{d \left[nF + nv \left(\frac{X}{n} \right)^m \right]}{dn} = F - (m-1)v \frac{X^m}{n^m} = 0.03 - 0.5(0.00001) \frac{X^m}{n^m} = 0.03 - \frac{113.14}{n^{1.5}} = 0$$

$$n = \sqrt[m]{\frac{(m-1)vX^m}{F}} = \sqrt[1.5]{\frac{0.5(0.00001) \cdot 80,000^{1.5}}{0.03}} = 242.28 \approx 242$$

- Thus, the optimal number of stock transactions for this order is $n = 242$, each with transaction size of X/n equal to $80,000/242 = 330$ (subject to rounding)

Improve the order split mechanism

- There are many factors that we can consider when designing the optimal cost or trade-slicing functions.
- The model above provides no guidance on spacing trades through the day or time frame
 - The amount of time elapsing between trades executing at time t and time $t+1$ would be inversely related to market impact costs. That is, more time elapses between the transactions, slippage will be reduced.
 - Stocks tend to be more liquid during the earliest and latest parts of the day, and least liquid in the middle. That is, one might expect liquidity during the day to be "U"-shaped, and market impact costs to resemble an upside down U.

2. Use of Limit Orders

- Suppose we want to buy 1000 shares
 - If we directly send a market order, the whole ask book will be cleared and the purchase price can be expensive
 - Instead of market order, we can place multiple limit orders at different price levels

Bid Size	Bid Price	Ask Price	Ask Size
20	9.9	10.5	100
30	9.6	11.0	50
50	8.8	11.6	300
50	8.1	12.3	100
10	7.5	12.9	120
50	7.3	20.0	200
10	7.1		
70	6.9		
80	6.6		

Current order book



Bid Size	Bid Price	Ask Price	Ask Size
250	10	10.5	100
20	9.9	11.0	50
30	9.6	11.6	300
250	9.5	12.3	100
250	9.0	12.9	120
50	8.8	20.0	200
250	8.5		
50	8.1		
10	7.5		
50	7.3		
10	7.1		
70	6.9		
80	6.6		

Potential issues of using limit orders

- Wait too long for the limit order to get filled if the limit price is set too far from the top book.
- Other market participants may discover the large order buyer/seller if there is a sudden increase in the order book's volume.
 - Solution: split into smaller limit order and place slowly to the market

3. Time-weighted Average Price (TWAP)

- Place market orders with the same size evenly over a specified time period.
- Parameters:
 - **total_order_size**: Total number of shares you want to buy or sell.
 - **execution_duration**: Total time over which to execute the order.
 - **interval**: Time interval for executing parts of the order.

$$\text{Shares per interval} = \text{Round}\left(\frac{\text{total_order_size}}{\text{execution_duration}/\text{interval}}, 0\right)$$

Example: TWAP

- Suppose we want to buy a total of 1000 shares.
- To reduce the market impact, we decide to buy a small fraction every 5 minutes for a total duration of 1 hour.
- Thus,
 - $\text{total_order_size} = 1000$
 - $\text{execution_duration} = 60$
 - $\text{interval} = 5$
- $\text{Shares per interval} = \text{round}(1000 / (60 / 5), 0) = 83$
- $\text{Remaining shares} = 1000 - 83 * (60 / 5) = 4$

Potential issues of TWAP

- Large orders executed over time can still impact market prices, especially for illiquid markets.
- Delays or failures in order execution at specified intervals (eg. due to market close) can disrupt the planned strategy.
- TWAP assumes a stable price environment; high volatility can lead to significant deviations.

4. Volume-weighted Average Price (VWAP)

- VWAP is the average price a security has traded at throughout the day, based on both volume and price.
- Formula

$$VWAP = \frac{\sum (Price \times Volume)}{\sum Volume}$$

Where

- Price: The price at which trades occur.
- Volume: The number of shares traded at each price.

Use of VWAP

- **Benchmarking:** Provides a standard for evaluating our purchase price.
- **Execution Strategy:** Helps traders execute orders with minimal market impact.
- **Trend Indicator:** Can signal potential reversals when the price diverges from VWAP.

VWAP Example

- Assume the following trades occur throughout a trading day:

Time	Price	Volume
09:30	\$100	200
09:35	\$101	150
09:40	\$102	250
09:45	\$100	300
09:50	\$103	100

- $VWAP = \frac{100*200+101*150+102*250+100*300+103*100}{200+150+250+300+100} = 100.95$
- So we can execute order in smaller increments around the VWAP to avoid impacting the market
- For example, you might place the following orders close to VWAP:
 - Buy 200 shares at \$100.90
 - Buy 300 shares at \$100.95
 - Buy 500 shares at \$101.00

Impact of VWAP on Execution

- By using VWAP:
 - The majority of orders are executed around the VWAP price, which is \$100.95.
 - The average execution price might be slightly higher than the VWAP, but it reduces the likelihood of large market movements caused by a single large order.

Order Book Modeling with Queuing Theory

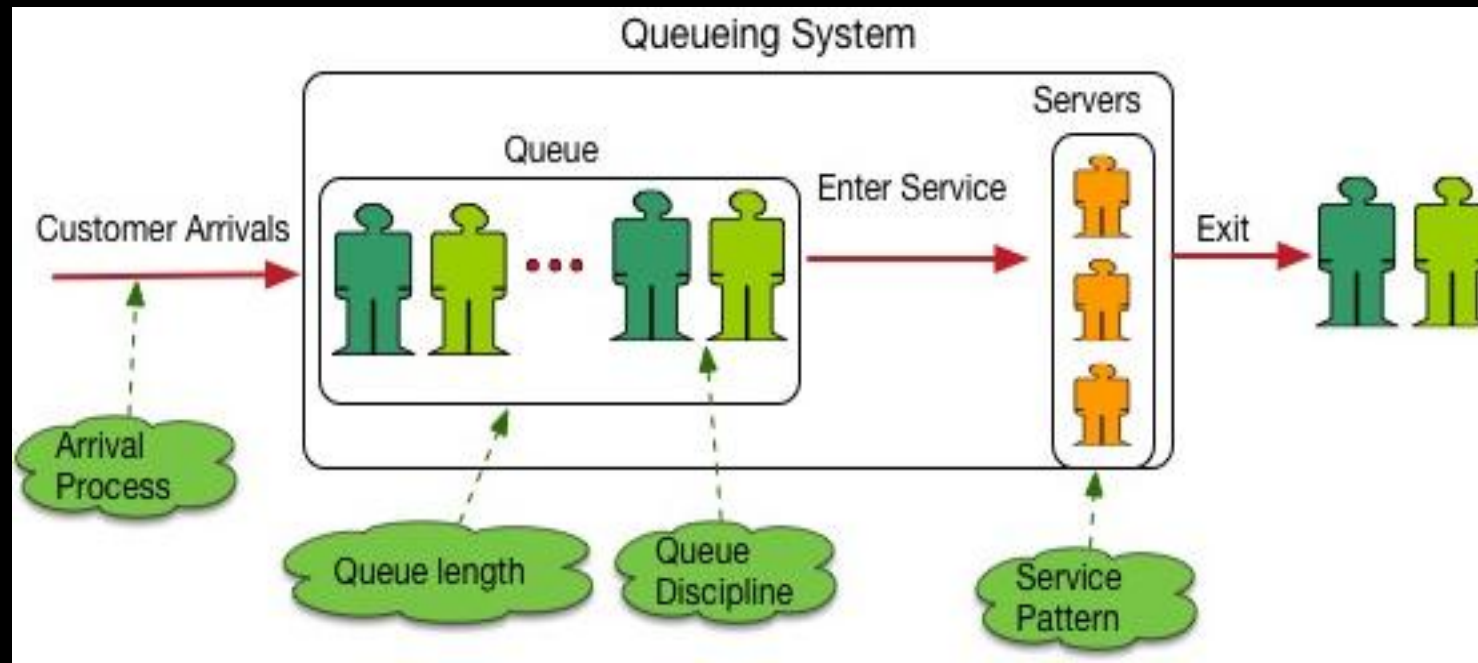
Why Queuing Theory?

- To analyze how orders flow through the market and how they impact price movement.
- Key Questions:
 - How long do orders wait before being executed?
 - What is the optimal order size?
 - How do market conditions (eg. volatility) affect execution time?

Basic Concepts of Queuing Theory

- Queue Components

- Arrival Process (λ): Rate at which an event arrive. (eg. customer)
- Service Rate (μ): Rate at which an event is executed.
- Queue Discipline: How events are processed (e.g. FIFO).



How is it related to Order Book?

- Incoming orders (buy/sell) can be viewed as entities arriving at a queue. The order book acts as a queue where these orders wait to be matched.
- Each order can be treated as a customer in a queue, where the "service" is the execution of trades.
- Arrival and Service Rates:
 - Arrival Rate: The rate at which orders enter the order book (eg. how many orders are placed per second).
 - Service Rate: The speed at which orders are matched and executed. This is influenced by the market's liquidity and the efficiency of the trading system.

How is it related to Order Book?

- **Event:** arrival of a new limit order
- **Queuing time:** the time to wait from placing a limit order until it reaches to the top book
- **Service time:** the elapsed time for the limit order being matched/executed when it is at the top book

Bid Size	Bid Price	Ask Price	Ask Size
20	9.9	10.5	100
30	9.6	11.0	50
50	8.8	11.6	300
50	8.1	12.3	100
10	7.5	12.9	120
50	7.3	13.0	1000
10	7.1	13.4	50
70	6.9		
80	6.6		



Bid Size	Bid Price	Ask Price	Ask Size
150	12.0	13.0	1000
150	11.5	13.4	50
100	10.8	13.6	200
100	10.5	14.3	150
20	9.9	14.9	100
30	9.6		
50	8.8		
50	8.1		
10	7.5		

Exponential Distribution

- Density function

$$f(x) = \begin{cases} \frac{1}{\lambda} e^{\frac{-x}{\lambda}}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

where λ is the average time taken between events.

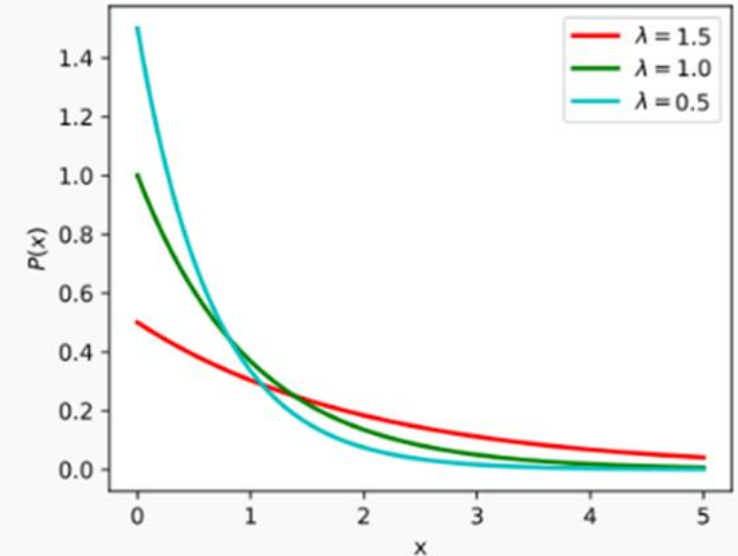
- Cumulative function

$$F(x) = \begin{cases} 1 - e^{\frac{-x}{\lambda}}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

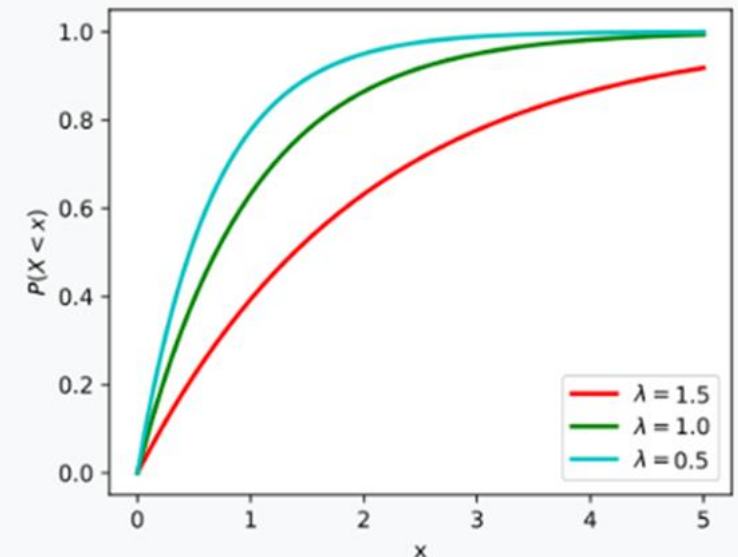
- Expected value $E(X) = \lambda$
- Variance $Var(X) = \lambda^2$
- Memorylessness property

$$P(X > s + t | X > s) = P(X > t)$$

Probability density function



Cumulative distribution function



Poisson Distribution

- Density function

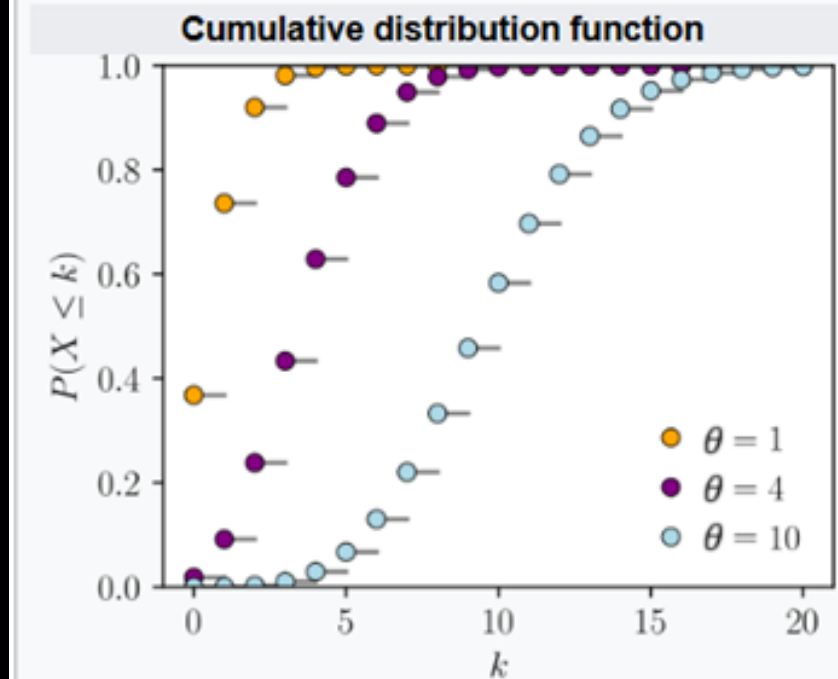
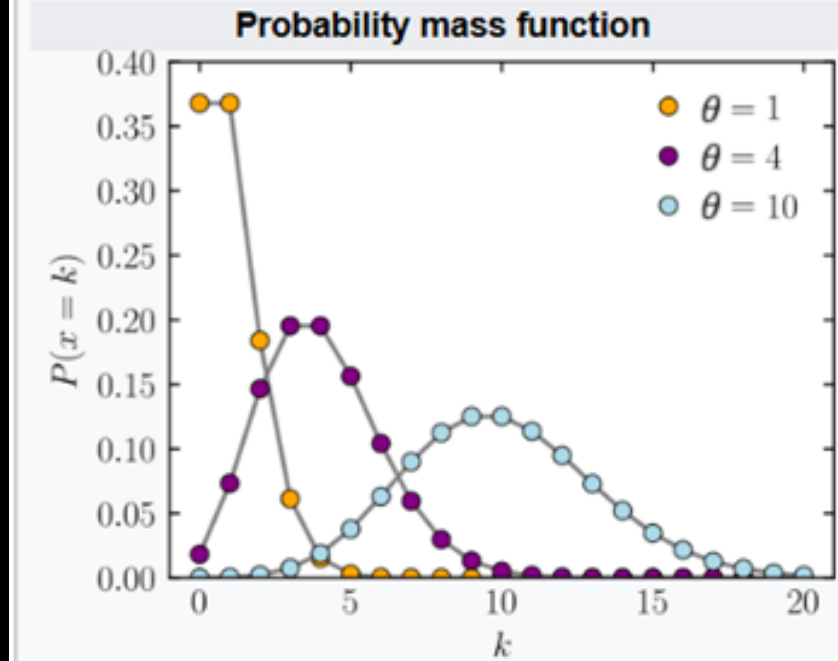
$$f(x) = \begin{cases} \frac{\theta^x e^{-\theta}}{x!}, & x = 0, 1, 2, \dots \\ 0, & \text{otherwise} \end{cases}$$

where θ is the average number of events occurred in a fixed time interval.

- Cumulative function

$$F(x) = \sum_{k=0}^x \frac{\theta^k e^{-\theta}}{k!}$$

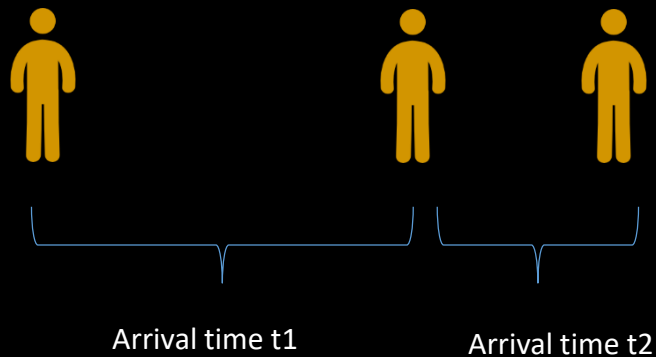
- $E(X) = Var(X) = \theta$



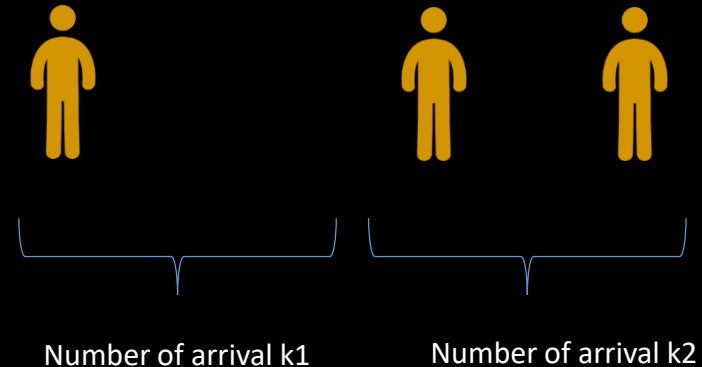
Poisson vs Exponential Distribution

- The Poisson distribution is used to count the number of events in a fixed time period, while the exponential distribution is used to model the time between these events.
- If events occur according to a Poisson process with rate θ , the time between consecutive events follows an exponential distribution with the rate λ . Then, we have

$$\theta = \frac{1}{\lambda}$$



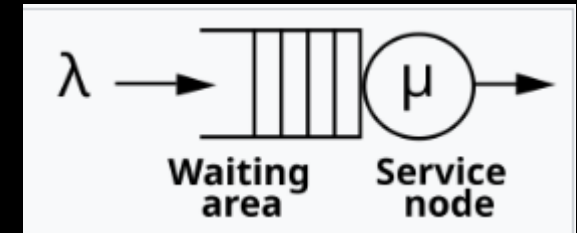
Exponential Distribution $f(t)$



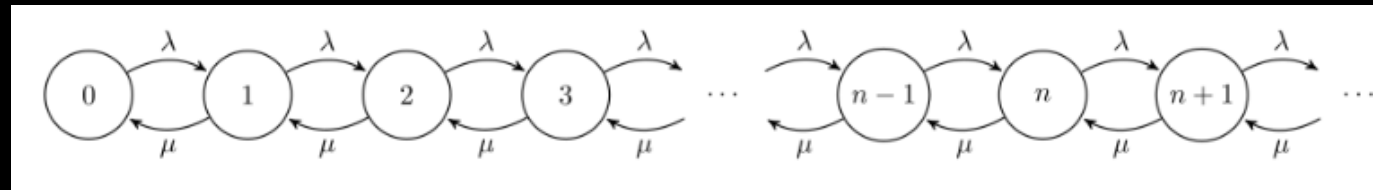
Poisson Distribution $f(k)$

One-side Queuing Model Formulation

- M/M/1 Queue:
 - M: Memoryless arrival process (Poisson)
 - M: Memoryless service process (Exponential)
 - 1: Single service channel (eg. one market maker)
- The model can be described as continuous time Markov chain with transition rate matrix on the state space $\{0,1,2,3,\dots\}$



$$Q = \begin{bmatrix} -\lambda & \lambda & & & \\ \mu & -(\mu + \lambda) & \lambda & & \\ & \mu & -(\mu + \lambda) & \lambda & \\ & & \mu & -(\mu + \lambda) & \lambda \\ & & & \ddots & \ddots \end{bmatrix}$$



Transition Rate Matrix

- In a typical M/M/1 queue transition rate matrix, the off-diagonal elements represent the rates of moving between states:
 - $Q_{i,i+1} = \lambda$: Transition from state i to $i+1$ (arrival).
 - $Q_{i,i-1} = \mu$: Transition from state i to $i-1$ (service completion).
- The diagonal entry $Q_{ii} = -(\lambda + \mu)$ refers to the total rate of leaving state i .
- The sum of each row in the transition rate matrix equals zero, meaning that the rate of leaving state i must equal the total rate of entering state i from all other states.

$$\sum_j Q_{ij} = 0$$

Steady-State Probabilities (π)

- To find the steady-state probabilities π of being in each state, we solve the system of equations derived from the balance equations $\pi Q = \mathbf{0}$ along with the normalization condition $\sum \pi_i = 1$.
 - π_i means the probability of having i events (i.e. limit orders) in the queuing system
- From the steady-state probabilities, we can derive various useful metrics:
 - Average number of customers in the system (L)
 - Average time a customer spends in the system (W)
 - Utilization of the server (ρ)

Queuing Model Formulation

- The model is considered stable only if $\lambda < \mu$. If, on average, arrivals happen faster than service completions the queue will grow indefinitely long and the system will not have a stationary distribution. The stationary distribution is the limiting distribution for large values of t .
- Under stationary process and first-come first-served (FIFO) discipline, the response time (i.e. the sum of waiting and service time) has a probability density function
 - i.e. the time for a new limit order entering into the order book until it is filled

$$f(t) = \begin{cases} (\mu - \lambda)e^{-(\mu - \lambda)t}, & t > 0 \\ 0, & \text{otherwise} \end{cases}$$

Traffic Intensity

- Traffic Intensity (ρ): $\rho = \frac{\lambda}{\mu}$
- Traffic intensity ρ measures the utilization of the queuing system.
- Ranges
 - $\rho < 1$: The system is stable which can handle incoming traffic without becoming overloaded. The queue will not grow indefinitely. Customers are served on average faster than they arrive.
 - $\rho = 1$: The system is operating at its capacity. The arrival rate equals the service rate, leading to a steady-state condition where the queue can grow without bound.
 - $\rho > 1$: The system is unstable. The arrival rate exceeds the service rate, leading to an ever-growing queue.

Key Results of M/M/1 Queuing Model

- Average time an order spends in the system (W):

$$W = E(T) = \frac{1}{\mu - \lambda} = \frac{1}{\mu(1 - \rho)}$$

- Average number of orders in the system (L):

$$L = \lambda W = \frac{\lambda}{\mu - \lambda} = \frac{\rho}{1 - \rho}$$

- Average time an order waiting in the queue (W_q):

$$W_q = W - \frac{1}{\mu} = \frac{\lambda}{\mu - \lambda} = \frac{\rho}{\mu(1 - \rho)}$$

Here $\frac{1}{\mu}$ represents the average service time.

- Average number of orders in the queue (L_q):

$$L_q = \lambda W_q = \frac{\rho^2}{1 - \rho}$$

Estimating λ (Arrival Rate)

- In trading, an "arrival" can refer to new limit orders entering the market.
- The arrival rate λ can be calculated as

$$\lambda = \frac{\text{Total number of arrivals}}{\text{Total time period}}$$

- Example: if a trading system receives 600 limit orders over a 10-minute period, λ would be

$$\lambda = \frac{600}{10} = 60 \text{ orders per minute}$$

Estimating μ (Service Rate)

- In trading, the "service" could refer to the execution of orders.
- The service rate μ can be calculated as:

$$\mu = \frac{1}{\text{Average service time}}$$

- Example: If the average time to execute an order is 2 seconds, then

$$\mu = \frac{1}{2} = 0.5 \text{ orders per second} = 30 \text{ orders per minute}$$

Example

- In an order book, buy orders arrive at an average rate of 12 orders per minute, while sell orders arrive at an average rate of 10 orders per minute.
- Assuming that both types of orders are processed immediately and that the system operates under a Poisson arrival process, what is the expected number of buy orders in the queue at any given time?
- Answer:
 - Arrival rate λ for buy orders = 12
 - Service rate $\mu = 12+10$ (i.e. the total rate at which orders can be processed)
 - Average number of orders in the system (L):

$$L = \frac{\lambda}{\mu - \lambda} = \frac{12}{22 - 12} = 1.2$$

- Average number of buy orders in the queue (L_q):

$$L_q = \frac{\lambda}{\mu} L = \frac{12}{22} 1.2 = 0.65$$

Remarks

- **Time Period:** Ensure that the time period for your data collection is consistent when calculating λ and μ .
- **Market Conditions:** Be aware that both λ and μ can vary significantly with market conditions (eg. volatility, liquidity).
- **Statistical Methods:** You can use statistical methods to refine your estimates, such as calculating confidence intervals or conducting regression analysis if you have larger datasets.

Key Assumptions of the Basic Queuing Model

- Poisson Arrivals:
 - Assumes that orders arrive randomly and independently.
 - Real-world order flows can exhibit clustering or trends.
- Exponential Service Times:
 - Assumes a constant average execution time for orders.
 - In reality, execution times can vary significantly due to order size, market volatility, etc.
- Single Server:
 - Assumes a single market maker or execution venue.
 - Multiple venues and decentralized markets can complicate execution dynamics

Trading Strategies based on Queuing Theory

1. Optimal Order Placement

- **Concept:** Apply queuing principles to optimize order placement.
- **Strategy:**
 - Analyze the average waiting times and queue lengths to determine the best times to place orders.
 - Use limit orders strategically when the queue is shorter (lower waiting times), and market orders when the queue is longer.
 - Adjust order sizes based on the current traffic intensity (ρ) to avoid slippage.

Trading Strategies based on Queuing Theory

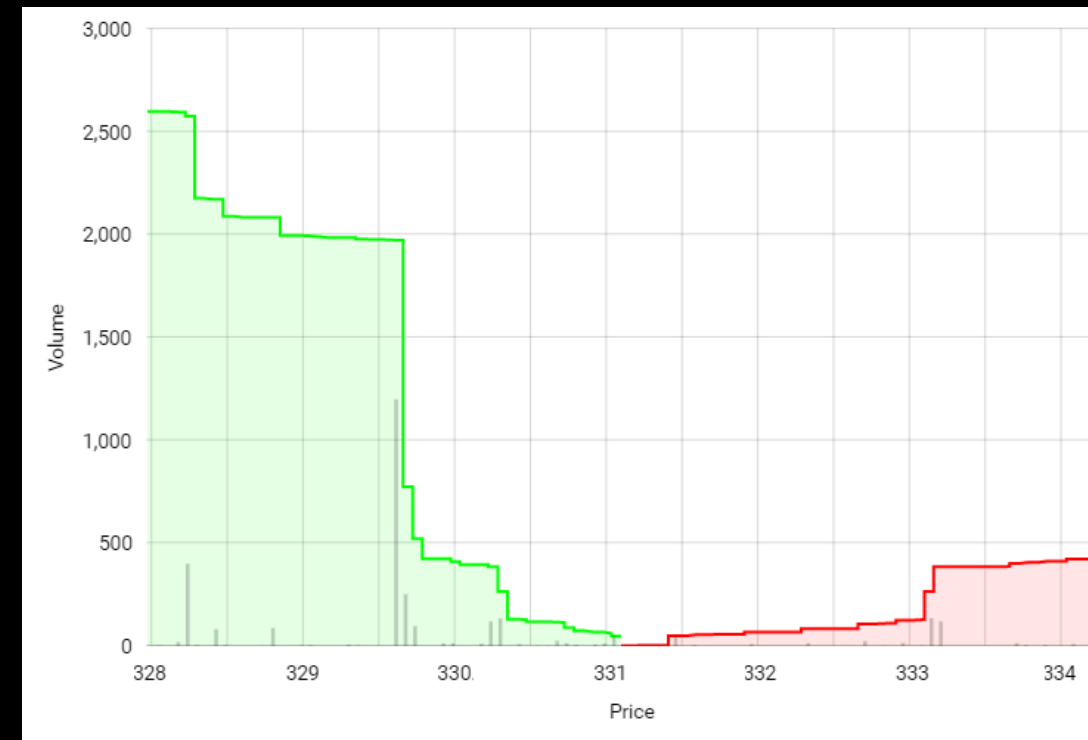
2. Order Flow Analysis

- **Concept:** Use arrival rates (λ) and service rates (μ) to analyse market liquidity.
- **Strategy:**
 - Monitor the number of incoming orders and execution times to gauge market activity.
 - Increase trading frequency during periods of high arrival rates to capitalize on liquidity.
 - Reduce trading during low liquidity periods.

Trading Strategies based on Queuing Theory

3. Queue Analysis

- **Concept:** Utilize queue lengths to inform directional move.
- **Strategy:**
 - When the queue length (or the area) of bid book is much longer than that of ask book, it may imply a strong market demand and suggest a short-term upward trend
 - Pair this with indicators that signal buy or sell conditions.



Backtest Order Book Imbalance

Trading Logic:

- For each second,
 - Collect the latest bid-book and ask-book data
 - Compute the total volume in both bid-book and ask-book respectively
 - Market Entry:
 - If last trade time is 15 minutes before,
 - If bid-book volume > 2* ask-book volume, open a BUY order
 - If ask-book volume > 2* bid-book volume, open a SELL order
 - Market Exit:
 - Close if previously open a BUY order, but now bid-book volume < ask-book volume
 - Close if previously open a SELL order, but now ask-book volume < bid-book volume

Backtest Settings

- Backtest Period = 2024-01
- Data Interval = Tick
- Initial Capital = 10,000 USD
- Leverage = 5
- Transaction Cost = 0
- Allow Short Sell = True
- Position Netting = False
- Instruments = BTCUSDM

Settings

Strategy Name

Order Imbalance

Start Period *

2024-01

End Period *

2024-01

Data Interval *

tick

Initial Capital *

10000.0

Base Currency *

USD

Leverage

5.0

Transaction Cost

0.0

Risk free Rate

0.0

Enable Short Sell

☒

Position Netting

☐

News Data

☐

Weather Data

☐

Economics Data

☐

Instruments ⓘ
(Max. 100 items)

BTCUSDM ✕



Full Script

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
from datetime import datetime, timedelta

class AlgoEvent:
    def __init__(self):
        self.timer = datetime(1970,1,1)
        self.last_trade = datetime(1970,1,1)
        self.thre = 2

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()

    def on_marketdatafeed(self, md, ab):
        # reset grid every 1 minute
        if md.timestamp >= self.timer+timedelta(seconds=1):
            # update timer
            self.timer = md.timestamp

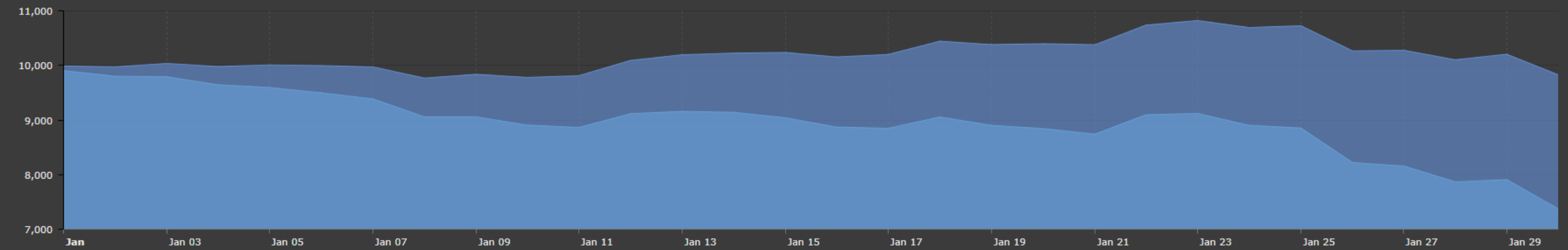
        # calculate cumulative volume for bid and ask book
        bid_volume, ask_volume = 0, 0
        for p,v in md.bidOrderBook:
            bid_volume +=v
        for p,v in md.askOrderBook:
            ask_volume +=v
```

```
# entry signal
if md.timestamp>=self.last_trade+timedelta(minutes=15):
    if bid_volume>self.thre*ask_volume:
        self.last_trade = md.timestamp
        order = AlgoAPIUtil.OrderObject(
            instrument=md.instrument,
            openclose='open',
            buysell=1,
            volume=0.0001
        )
        self.evt.sendOrder(order)
    elif ask_volume>self.thre*bid_volume:
        self.last_trade = md.timestamp
        order = AlgoAPIUtil.OrderObject(
            instrument=md.instrument,
            openclose='open',
            buysell=-1,
            volume=0.0001
        )
        self.evt.sendOrder(order)

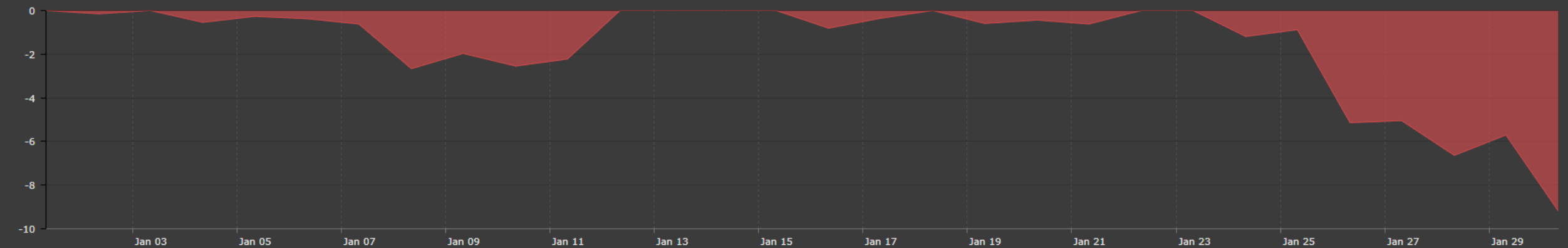
# exit signal
pos, OrderList, LmtOrderList = self.evt.getSystemOrders()
for tradeID in list(OrderList):
    buysell = OrderList[tradeID]['buysell']
    if (buysell==1 and bid_volume<ask_volume) or (buysell==-1 and ask_volume<bid_volume):
        order = AlgoAPIUtil.OrderObject(
            tradeID=tradeID,
            openclose='close'
        )
        self.evt.sendOrder(order)
```

Backtest Result

Capital Usage:



Equity Drawdown (%):



Performance Statistics: ⓘ

No. of Tradable Days:	31	No. of Win Days:	16	No. of Loss Days:	14
Win Rate:	53.3333%	Max. Consecutive Win Day:	5	Max. Consecutive Loss Day:	3
Odd Ratio:	1.1429	Max. Consecutive Gains:	457.88	Max. Consecutive Loss:	-461.27
No. of Trades:	2919	Average Consecutive Win Day:	0.93	Average Consecutive Loss Day:	0.6
Total PnL:	-173.62	Average Per Trade PnL:	-0.06	Average Per Day PnL:	-5.6
Mean Daily Return:	-0.0462%	Median Daily Return:	0.1058%	Mean Annual Return:	-11.6511%
Daily Return StdDev:	1.5529%	25th percentile Daily Return:	-0.5793%	75th percentile Daily Return:	0.5763%
Daily Return Downside StdDev:	1.0594%	95% 1 day return VaR:	-2.9652%	99% 1 day return VaR:	-4.128%
Daily Sharpe Ratio:	-0.0298	Annual Sharpe Ratio:	-0.4726	Max. Drawdown Amount:	995.9
Daily Sortino Ratio:	-0.0436	Annual Sortino Ratio:	-0.6928	Max. Drawdown Percent:	9.2023%
Max. Drawdown Duration:	8	Average Drawdown Duration:	2.45	Annual Volatility:	24.5542%
Gross Profit:	0.0	Gross Loss:	0.0	Profit Factor:	0.0
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treynor Ratio:	0.0	Tail Ratio:	0.8897
Calmar Ratio:	1.2661	Average Holding Day:	14.4259	Annual Turnover Rate:	0.0%

Limitations of the Basic M/M/1 Queuing Model

- Assumes a simple FIFO processing, while actual order execution may depend on priority rules or other factors. Eg. a trader can randomly place a limit order at any price level.
- Market makers may keep placing and cancelling limit orders which makes the analysis more complicated.
- It ignores the size of order. A larger size is expected to take longer time to get filled (i.e. longer service time)
- Assumes constant arrival and service rates, while these rates can fluctuate with market volatility and liquidity

Optional Reading

- The order book as a queueing system: average depth and influence of the size of limit orders (<https://hal.science/hal-01006410/document>)
- Simulating and analyzing order book data: The queue-reactive model (<https://arxiv.org/pdf/1312.0563>)
- A Model for Queue Position Valuation in a Limit Order Book (<https://moallemi.com/ciamac/papers/queue-value-2016.pdf>)

High Frequency Trading



VK 影視

天下武功，无坚不破，唯快不破

In the world of kung fu, speed defines the winner.

High Frequency Trading

- Rapid executions of multiple transactions followed by extremely short holding periods.
 - Profit from small price changes and discrepancies
 - Require sophisticated technology, communications and computing resources
- High frequency trading depends heavily on execution speed and latency

Co-location

- High frequency trading firms usually rent space from market centers, such as from the major exchanges
- The purpose to put their servers next to the market center's data servers is to reduce latency. It provides significant millisecond and even microsecond advantages, and enable institutions to better serve their clients.



Co-location

- Light or an electronic signal, travels approximately 300 meters in just 1 microsecond. Therefore,
 - from the NYSE to a financial institution in Hong Kong, which is around 13,000 kilometers away, it would take about 43.3 milliseconds
- This calculation ignores the additional latency induced by routers, switches, etc.
- An exchange provides institutions that co-locate in its facilities with cables of identical length to connect its data feeds, regardless of how many meters an office is from the feeds.

Latency Arbitrage

- Latency refers to the speed at which quotations are displayed or orders are executed after being placed
- Consider this example involving a high-frequency trader (HFT) that uses automated trading to leverage its informational edge:
 - The initial market bid and ask price for Stock X are 50.00 and 50.01.
 - Due to its advanced technology and strategic location, HFT receives an institutional market order for Stock X. This information reaches the trader's system 10 microseconds before it does for most competing traders.
 - HFT purchases all available shares at 50.01 before other traders can react.
 - As a result, the institutional buyer is unable to buy at 50.01.
 - HFT then place a sell limit order at 50.02, and thus market ask price adjusts to 50.02.
 - The institutional buyer's market order finally execute at 50.02, locking a 0.01 profit per share for HFT

Latency Arbitrage

- Another example highlighted in the Wall Street Journal.
 - In July 2009, Intel, announced strong earnings, prompting some traders to view the semiconductor company Broadcom as a potential buying opportunity.
 - To mask their interest, many of these traders divided their orders into smaller quantities.
 - On that day, Broadcom opened trading at \$26.20. High-frequency traders (HFTs), who had quicker access to market quotes, noticed the significant interest in Broadcom.
 - Consequently, the HFTs started purchasing Broadcom shares and subsequently reselling them to slower investors at elevated prices. The share price of Broadcom swiftly climbed to \$26.39 as HFTs began offering hundreds of thousands of shares for sale.
 - Slower traders ended up spending \$1.4 million for around 56,000 shares, incurring an additional cost of \$7,800 compared to if they had been able to act as quickly as the high-frequency traders.

Common High Frequency Strategy

1. **Market Making**

- Providing liquidity by placing buy and sell orders simultaneously to profit from the bid-ask spread
- Some exchanges also provide rebates to market makers

2. **Latency Arbitrage**

- Exploiting delays in market data transmission to execute trades before other market participants can react
- Co-location for rapid data feed and order execution is crucial

3. **Cross Market Arbitrage**

- Identifying and exploiting price discrepancies between different asset classes, markets or exchanges. (eg. 0700.HK vs TCEHY)

4. **Order Flow Analysis**

- Monitoring the flow of buy and sell orders to predict short-term market direction.

HFT: Is it worth?

- It is an arms race to build
 - more reliable and faster execution platforms (computer engineering and science)
 - more comprehensive and accurate prediction models (mathematics, physics)
 - Reduce the latency (information engineering)
- It is a money burning game to buy or rent
 - Faster data feed and lower latency
 - Bigger storage space and more powerful system
- But, after you spend so much money, can you make the profit?



高頻交易 | 傳內地打壓高頻交易 要求將客戶伺服器遷出交易所數據中心

BossMind

2026年1月16日



彭博引述知情人士報道，中國監管機構要求將客戶伺服器遷出交易所運營的數據中心，料將削高頻交易機構的優勢。據報上海和廣州的大宗商品期貨交易所，已依照監管機構指示，要求本地經紀公司將客戶伺服器遷出交易所營運的數據中心。

知情人士透露，這一舉措由監管機構主導，上海期貨交易所已告知經紀商，他們需要在下個月底前將高頻客戶的設備移出，而其他客戶則需要在4月30日前完成。

這一措施影響的不僅僅是高頻交易機構，但它們受到的衝擊可能最大。除了內地本土的高頻交易公司，城堡證券（Citadel Securities）、Jane Street Group和Jump Trading等外國公司的伺服器接入亦將受到影響。

Arbitrage

What is arbitrage?

- Arbitrage is the simultaneous purchase and sale of related assets in different markets to profit from price discrepancies.
- It usually requires efficient algorithms and high performing computers for execution.
- Helps in maintaining market efficiency by aligning prices across different markets.

Type of Arbitrage

1. Spatial Arbitrage

- Involves buying and selling the same asset in different locations.

2. Temporal Arbitrage

- Involves buying and selling the same asset at different times.

3. Statistical Arbitrage

- Relies on mathematical models to identify price inefficiencies.

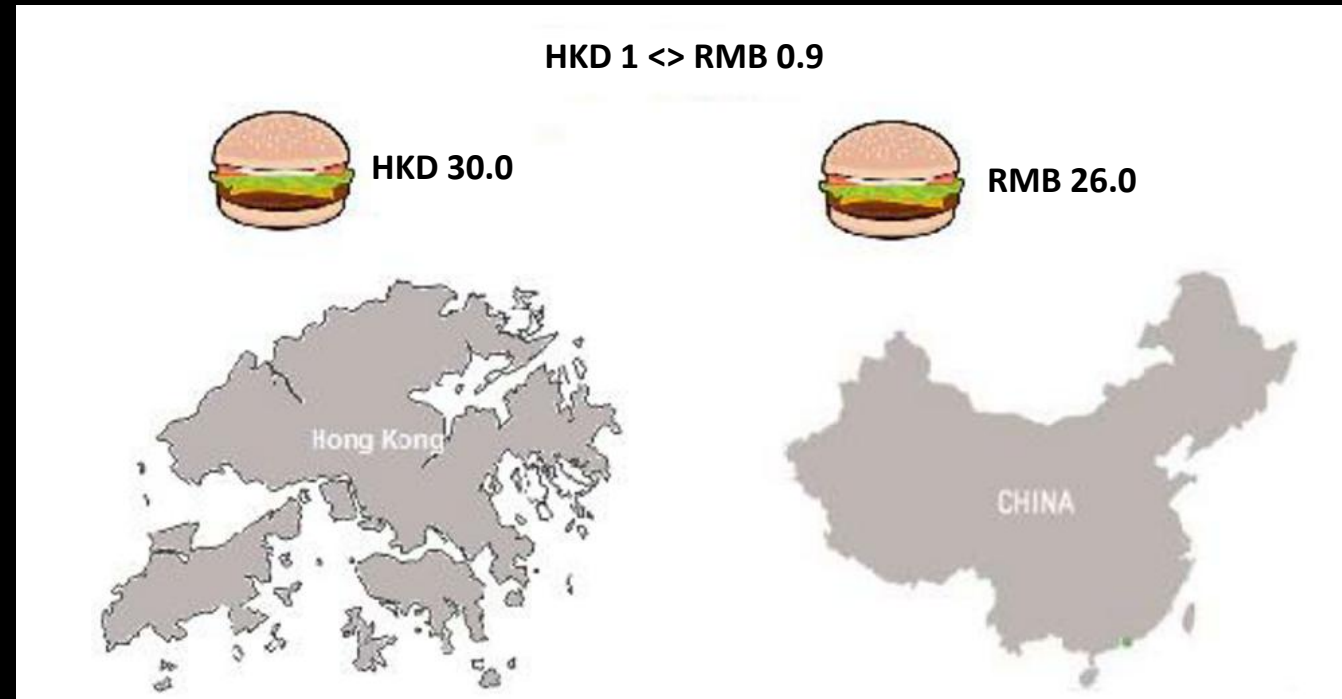
Spatial Arbitrage: Big Mac Example

Suppose

- Big Mac in Hong Kong costs 30 HKD
- Big Mac in Shenzhen costs 26 RMB
- Exchange rate: 1 HKD = 0.9 RMB

Actions:

- Buy a Big Mac in Shenzhen
- Sell it in Hong Kong
- Net Profit = $30 \times 0.9 - 26 = 1$ RMB



Spatial Arbitrage: Cross Exchanges Example

- Suppose you have US\$100,000 in both Binance and Bybit account.
- Let's say you observe the following bitcoin future prices:

Exchange	Bid Price	Ask Price
Binance	96,531.5	96,532.0
Bybit	96,529.0	96,531.0

- Actions:
 - Buy 1 contract of bitcoin future on Bybit at \$96,531
 - Sell 1 contract of bitcoin future on Binance at \$96,531.5
 - Net Profit = \$0.5

Temporal Arbitrage Example

- Suppose 2 companies A (acquiring company) and B (target company) are going to merge to company C.
- After merge, A and B will be removed from stock exchange while C will be listed.
- Before the merge is done
 - the shares value of the targeted company B will typically be lower than that of the combined company C
 - the share value of the acquiring company A will be slightly higher than that of the combined company C
- An arbitrageur can buy shares from the targeted company and sell shares for the acquiring company at the same time. Upon the merger is closed, the discrepancy in shares value will be the profit.

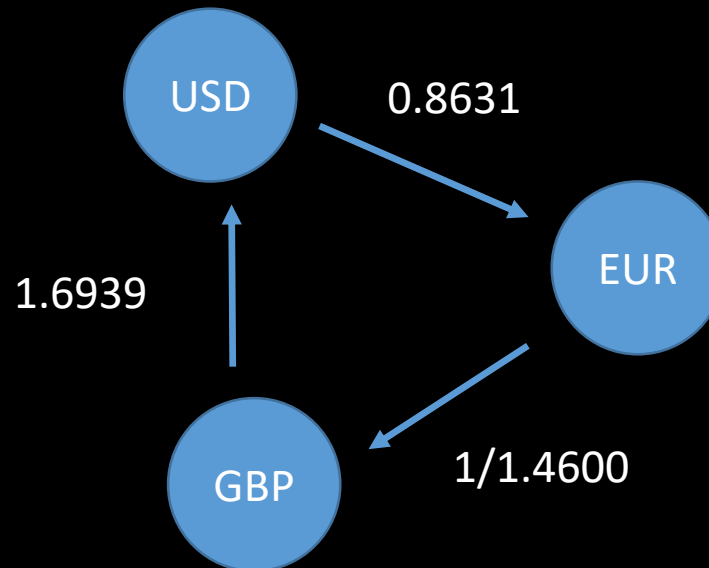
High Frequency Strategy 1: Triangular Arbitrage

Triangular Arbitrage

- Triangular arbitrage is a trading strategy that exploits discrepancies in currency exchange rates.
- Suppose you have US\$1000 and observe the current exchange rates in the market
 - EUR/USD = 0.8631
 - EUR/GBP = 1.4600
 - USD/GBP = 1.6939
- Note:
 - In market convention, the quotation XXX/YYY means the amount of XXX you can get in exchange for 1 unit of YYY

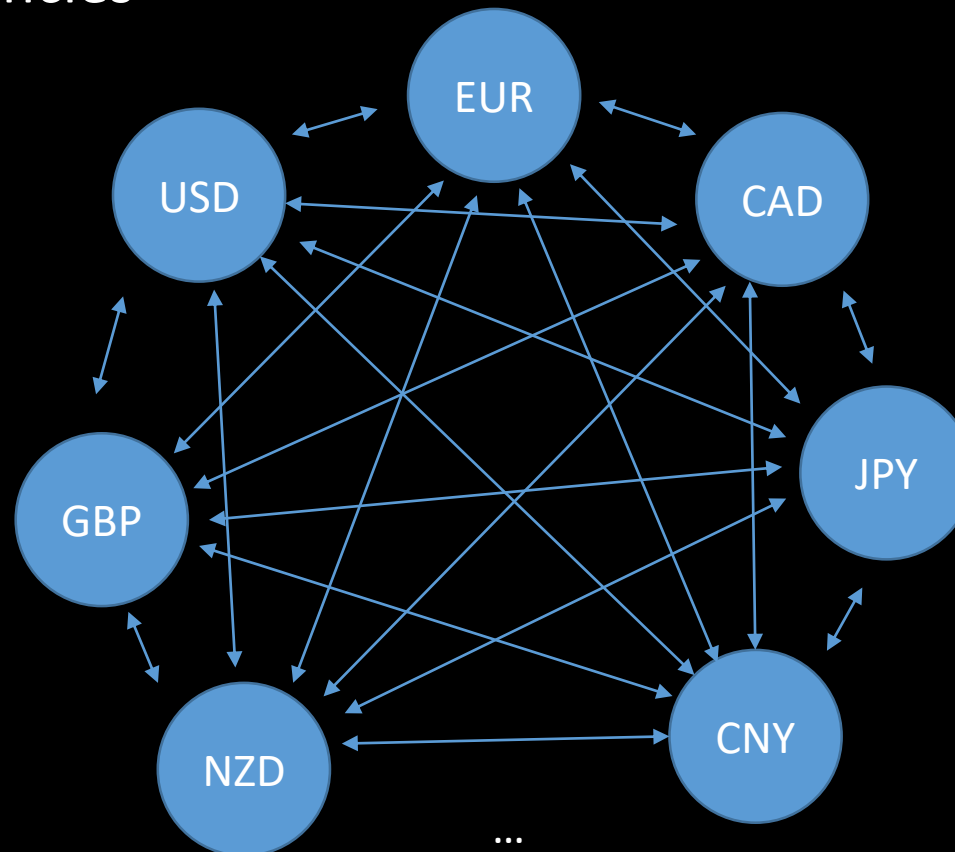
Triangular Arbitrage

- The exchange rates do not match up. We can capture the arbitrage profit as follows:
 - Sell dollars for euros: $\$1000 \times 0.8631 = \text{€}863.1$
 - Sell euros for pounds: $\text{€}863.1 / 1.4600 = \text{£}591.1644$
 - Sell pounds for dollars: $\text{£}591.1644 \times 1.6939 = \1001.373
 - Net profit = $\$1001.373 - \$1000 = \$1.373$



Polygonal Arbitrage

- The idea of Triangular Arbitrage can be extended to a so-called “Polygonal Arbitrage” among N currencies



Polygonal Arbitrage - Example

- Suppose we obtain the following snapshot at a particular time during a trading day.

Symbol	Bid	Ask
CAD/AUD	0.89068	0.89071
CNY/AUD	4.67769	4.67784
EUR/AUD	0.61055	0.61065
GBP/AUD	0.5178	0.51791
JPY/AUD	73.5981	73.6017
NZD/AUD	1.04152	1.04163
USD/AUD	0.66956	0.66964
CNY/CAD	5.24968	5.25003
EUR/CAD	0.68523	0.6853
GBP/CAD	0.58144	0.58161
JPY/CAD	82.5996	82.6034
NZD/CAD	1.16878	1.16898
CAD/USD	1.33033	1.33037
EUR/CNY	0.13051	0.13055

Symbol	Bid	Ask
GBP/CNY	0.11074	0.11075
JPY/CNY	15.7357	15.7364
NZD/CNY	0.22269	0.22277
CNY/USD	6.98578	6.98589
GBP/EUR	0.84851	0.84853
JPY/EUR	120.558	120.562
NZD/EUR	1.70653	1.70667
USD/EUR	1.09665	1.09667
JPY/GBP	142.092	142.113
NZD/GBP	2.01151	2.01182
USD/GBP	1.29277	1.2929
JPY/NZD	64.7773	64.7779
JPY/USD	109.924	109.931
USD/NZD	0.64288	0.64298

Polygonal Arbitrage - Example

- We need to take into account of the bid-ask direction.
- For a quotation XXX/YYY,
 - Use ask price for calculation if we convert from XXX to YYY
 - Use bid price for calculation if we convert from YYY to XXX
- Eg. JPY/USD,
 - If you sell 100 JPY for USD (i.e. buy USD), you will convert using ask and get $100/109.931 = 0.91$ USD.
 - If you sell 100 USD for JPY (i.e. buy JPY), you will convert using bid and get 10,992.4 JPY

Symbol	Bid	Ask
JPY/USD	109.924	109.931

Polygonal Arbitrage - Example

- We are interested in the following questions
 1. Is there arbitrage opportunity?
 2. What is the optimal path that can yield the highest return after a circulation?
- The most straightforward method is by using high school mathematics!!!
 1. Loop for any 3,4,5,... number of currencies (i.e. combination)
 2. For each combination, loop for different currency order (i.e. permutation)
 3. Multiply the exchange rate along the currency path
 4. Arbitrage exists if the value is greater than 1

Polygonal Arbitrage - Example

```
from itertools import combinations, permutations
```

```
def calculate_profit(rate_dict, currencies):
    initial_amount = 1 # Starting with 1 unit of the first currency
    current_amount = initial_amount
    action_path = []

    print('currencies=', currencies)

    for i in range(len(currencies)):
        from_currency = currencies[i]
        to_currency = currencies[(i + 1) % len(currencies)] # Wrap around

        if (to_currency, from_currency) in rate_dict:
            bid_price = rate_dict[(to_currency, from_currency)][0]
            current_amount *= bid_price
            action_path.append(f'Buy {current_amount:.5f} {to_currency} from {initial_amount:.4f} {from_currency} at bid price {bid_price:.5f}')
            initial_amount = current_amount # Update for the next trade

        if (from_currency, to_currency) in rate_dict:
            ask_price = rate_dict[(from_currency, to_currency)][1]
            current_amount /= ask_price
            action_path.append(f'Sell {initial_amount:.5f} {from_currency} for {current_amount:.5f} {to_currency} at ask price {ask_price:.5f}')
            initial_amount = current_amount # Update for the next trade

    return current_amount, action_path
```

```
def find_polygonal_arbitrage(exchange_rates): # Create a dictionary to store the
exchange rates
    rate_dict = {}
    for pair, bid, ask in exchange_rates:
        cur1, cur2 = pair.split('/')
        rate_dict[(cur1, cur2)] = (bid, ask)

    # Iterate through combinations of currencies (from 3 up to the number of
currencies available)
    currencies = set(pair.split('/')[0] for pair in [pair[0] for pair in exchange_rates])

    for r in range(3, len(currencies) + 1):
        for currency_combination in combinations(currencies, r):
            for currency_permutation in permutations(currency_combination):
                value, action_path = calculate_profit(rate_dict, currency_permutation)
                if value > 1: # More than the initial investment indicates arbitrage
                    return True, action_path, value
    return False, [], 0 # No arbitrage opportunity found
```

Polygonal Arbitrage - Example

```
currencies= ('JPY', 'CAD', 'NZD')
currencies= ('JPY', 'NZD', 'CAD')
Arbitrage Opportunity Exists:
Sell 1.00000 JPY for 0.0154373636687821 NZD at ask
price 64.77790
Sell 0.01544 NZD for 0.013205840706241425 CAD at ask
price 1.16898
Buy 1.09080 JPY from 0.0132 CAD at bid price
82.59960
Total value after circulation: 1.0908
```

```
exchange_rates = [
    ['CAD/AUD', 0.89068, 0.89071],
    ['CNY/AUD', 4.67769, 4.67784],
    ['EUR/AUD', 0.61055, 0.61065],
    ['GBP/AUD', 0.5178, 0.51791],
    ['JPY/AUD', 73.5981, 73.6017],
    ['NZD/AUD', 1.04152, 1.04163],
    ['USD/AUD', 0.66956, 0.66964],
    ['CNY/CAD', 5.24968, 5.25003],
    ['EUR/CAD', 0.68523, 0.6853],
    ['GBP/CAD', 0.58144, 0.58161],
    ['JPY/CAD', 82.5996, 82.6034],
    ['NZD/CAD', 1.16878, 1.16898],
    ['CAD/USD', 1.33033, 1.33037],
    ['EUR/CNY', 0.13051, 0.13055],
    ['GBP/CNY', 0.11074, 0.11075],
    ['JPY/CNY', 15.7357, 15.7364],
    ['NZD/CNY', 0.22269, 0.22277],
    ['CNY/USD', 6.98578, 6.98589],
    ['GBP/EUR', 0.84851, 0.84853],
    ['JPY/EUR', 120.558, 120.562],
    ['NZD/EUR', 1.70653, 1.70667],
    ['USD/EUR', 1.09665, 1.09667],
    ['JPY/GBP', 142.092, 142.113],
    ['NZD/GBP', 2.01151, 2.01182],
    ['USD/GBP', 1.29277, 1.2929],
    ['JPY/NZD', 64.7773, 64.7779],
    ['JPY/USD', 109.924, 109.931],
    ['USD/NZD', 0.64288, 0.6429]
```

```
]
has_arbitrage, action_path, value = find_polygonal_arbitrage(exchange_rates)
if has_arbitrage:
    print("Arbitrage Opportunity Exists:")
    for action in action_path:
        print(action)
    print(f"Total value after circulation: {value:.4f}")
else:
    print("No Arbitrage Opportunity.")
```

Computational Complexity

- Combinations
 - The number of combinations of n currencies taken r at a time is given by $C(n,r)$ which is $\frac{n!}{r!(n-r)!}$
 - This operates in $O(n^r)$ for each combination size r
- Permutations
 - The number of permutations of r currencies is $r!$
 - For each combination, we need to consider all permutations, which adds an additional factor of $r!$
- Total Complexity
 - For each combination of currencies, we compute the profit for all permutations. Therefore, the total computational complexity can be approximated as: $O(\sum_{r=3}^n C(n,r) \cdot r!)$
 - This can be simplified to $O(n! \cdot 2^n)$ in the worst case because you are iterating through all combinations and permutations of the currencies.

Important Notes

To successfully implement the polygonal arbitrage strategy,

- Take into account of the bid-ask prices
- Factor in transaction cost in the calculation
- Beware of the rounding issue
- Beware of market slippage
- Lower chance of arbitrage if the bid-ask spread is large
- Need a fast-computing algorithm to solve for a path with
 - The highest return (global maximum), or
 - The first identified arbitrage

Can you execute in real market?

1. Cash exchange physically

- Compare prices across different shops
- Need manually input price in your program for calculation
- Slow processing and settlement due to human operation

NO



Can you execute in real market?

2. Currency exchanges via banks

- Bid-ask spread is too large
- How can you to get the real-time exchange rates and trade programmatically on the banking system?

NO

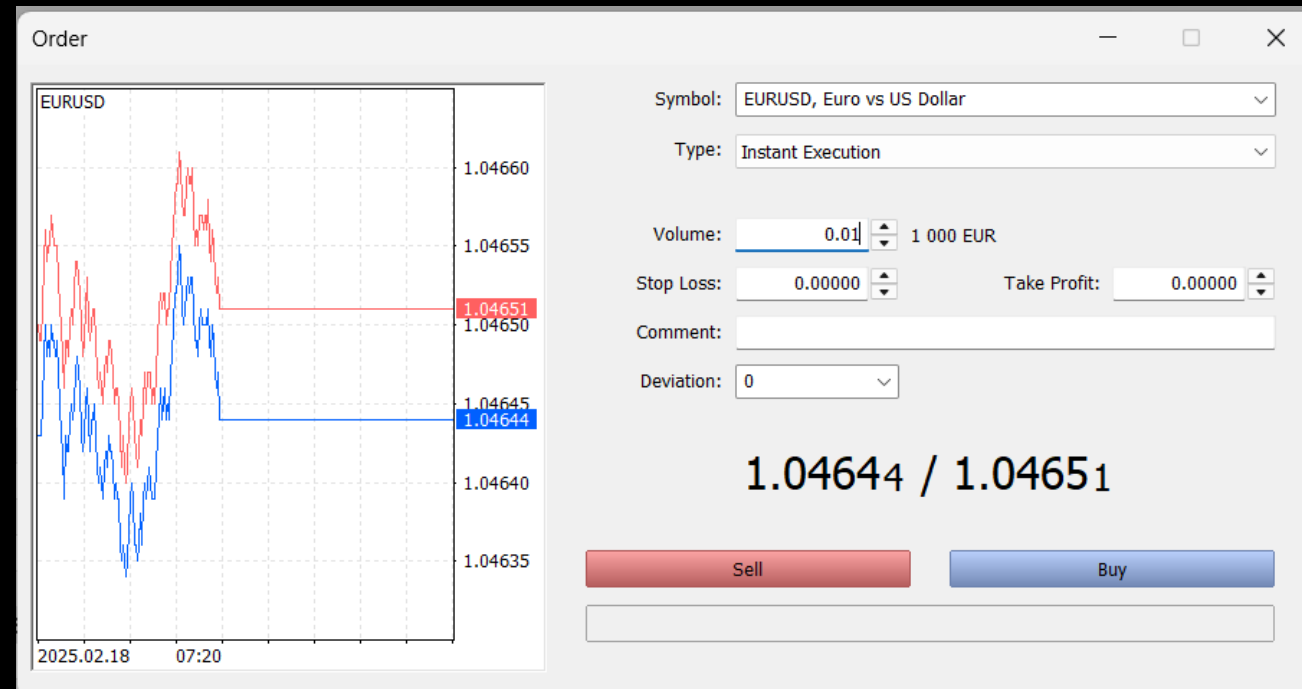
 Banking Accounts & Services Borrowing Cards & Loans Investing Securities & Currency Exchange Insurance Protection & Planning Insights Analysis & Market Data Offers Latest Rewards					
Exchange rates of foreign currencies against HK Dollar					
Currency	Telegraphic Transfer Bank Buy	Telegraphic Transfer Bank Sell	Banknotes Bank Buy	Banknotes Bank Sell	Last Updated
US Dollar	7.74820	7.80980	7.71630	7.84170	as at 18 Feb 2025 13:28 HKT
Australian Dollar	4.90450	4.97840	4.87720	5.00560	as at 18 Feb 2025 13:28 HKT
Canadian Dollar	5.43200	5.51860	5.40080	5.54980	as at 18 Feb 2025 13:28 HKT
Euro	8.06410	8.20180	8.02470	8.24120	as at 18 Feb 2025 13:28 HKT
Japanese Yen	0.05076	0.05154	0.05041	0.05189	as at 18 Feb 2025 13:28 HKT
New Zealand Dollar	4.39730	4.47870	4.37250	4.50400	as at 18 Feb 2025 13:28 HKT
Pound Sterling	9.71900	9.87600	9.61600	9.97800	as at 18 Feb 2025 13:28 HKT
Renminbi	1.06170	1.07590	1.05570	1.08190	as at 18 Feb 2025 13:28 HKT
Singapore Dollar	5.74340	5.83060	5.70300	5.87100	as at 18 Feb 2025 13:28 HKT
Swiss Franc	8.55460	8.67190	8.50010	8.72650	as at 18 Feb 2025 13:28 HKT
Thai Baht	0.22660	0.23420	0.21690	0.24390	as at 18 Feb 2025 13:28 HKT

Can you execute in real market?

3. FX brokers (eg. via MetaTrader)

- It is CFD trading, not currency exchange!
- But it might still work...

MAY BE



Triangular Arbitrage for CFD market

- Recall that triangular arbitrage exists if

$$\frac{EUR}{USD} \frac{USD}{JPY} \frac{JPY}{EUR} > 1$$

- It can be re-written as

$$\log\left(\frac{USD}{EUR}\right) = \log\left(\frac{USD}{JPY}\right) + \log\left(\frac{JPY}{EUR}\right) + \varepsilon \quad , \text{ where } \varepsilon \text{ is an error term very close to zero}$$

- We can fit a multiple linear regression (without intercept).

$$Y_t = \beta_1 X_{1,t} + \beta_2 X_{2,t} + \varepsilon_t$$

- The estimated value of β_1 and β_2 will be very close to 1.
- It can be traded as a mean reversion strategy based on the price discrepancies.

Can you execute in real market?

- We need a broker that supports FX Spot trading or currency exchange (eg. Interactive Brokers, Futu, etc)
- IB charges at least 2 USD commission per transaction
- For a circulation involving 3 currencies, total cost = 6 USD.
- Suppose our program detects 0.01% arbitrage profit among 3 currencies, the breakeven capital will be $\frac{6}{0.01\%} = 60,000$ USD
- If we require the cost ratio to be 10%, we need capital at least $\frac{60,000}{10\%} = 600,000$ USD

MAY BE

InteractiveBrokers	
Monthly Trade Value (USD) ²	Tiered
≤ 1,000,000,000	0.20 basis point ³ * Trade Value ⁴
1,000,000,000.01 - 2,000,000,000	0.15 basis point ³ * Trade Value ⁴
2,000,000,000.01 - 5,000,000,000	0.10 basis point ³ * Trade Value ⁴
> 5,000,000,000	0.08 basis point ³ * Trade Value ⁴
Minimum per order ²	Tier I - USD 2.00 Tier II - USD 1.50 Tier III - USD 1.25 Tier IV - USD 1.00

Conclusion

- FX market is quite efficient nowadays.
 - Triangular/Polygonal arbitrage doesn't occur very often.
 - Arbitrage profit, if any, its return won't be too high.
- However, it requires
 - a large capital to cover the potential costs
 - an efficient algorithm (software) and fast computer (hardware) to execute the trades
- It is not wise to put so much idle money to wait for small profits
- Thus, not many market players still use this strategy

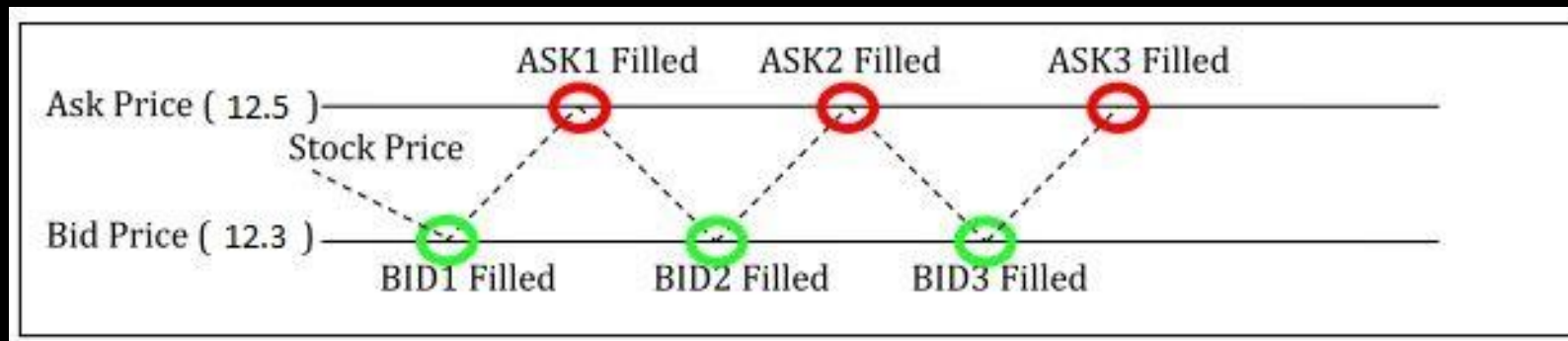
High Frequency Strategy 2: Market Making

Liquidity Providing

- Some high frequency traders can make profits from order flow.
- Exchanges usually offer rebates (eg. \$0.0025 per share) to certain brokers (so called “market maker”) that place limit orders in the market.
- The rebate is a payment to encourage brokers to provide liquidity in the market.
- A broker places a buy limit order that filled at \$10, and quickly places another sell limit order that filled again at \$10, can still make a \$0.005 profit per share from the liquidity rebate.
- Other buy side traders may pay higher fee to exchange, because it is regarded as the cost of using liquidity in the market.

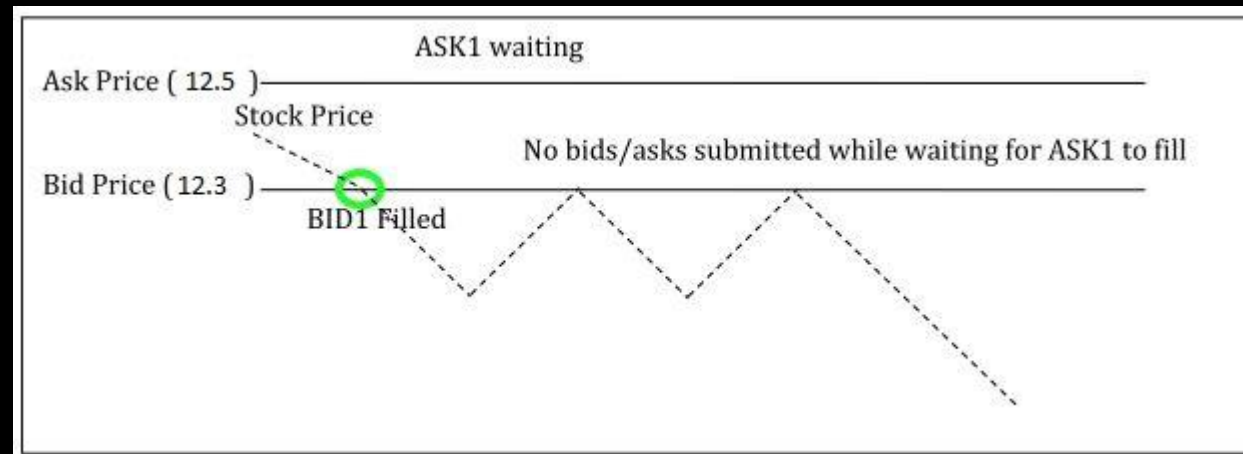
Market Making Strategy

- Market making strategy generates profits by capturing the bid-ask spread
- The simplest execution of a market making strategy is to place a pair of bid and offer trades, and have the 2 trades filled over time.
- When the market is not in trending, this algorithm is reasonably effective to earn the price differential.



Market Making Strategy

- However, this strategy fails if the market does not stay in a trading range, because one of the orders doesn't get fill and wait for a long time to capture the spread.
- A logical solution to this is, if a pair of order doesn't get fill on both side, we would want to reset the algorithm and submit a new set of pairs:
 - Constantly check if we have unfilled orders in the order book.
 - If no, we can simply submit a new set of pairs.
 - Otherwise, we should cancel the previous unfilled order, and submit another pair of limit orders.
 - Try to adjust the bid-ask level or quantity so as to balance the inventory.



Market Making Strategy Backtest

- Backtest Period = 2024-01 to 2024-03
- Data Interval = 1 hour
- Initial Capital = 100,000 USD
- Leverage = 1
- Transaction Cost = 0
- Allow Short Sell = True
- Position Netting = True
- Instruments = XAUUSD

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
from datetime import datetime, timedelta

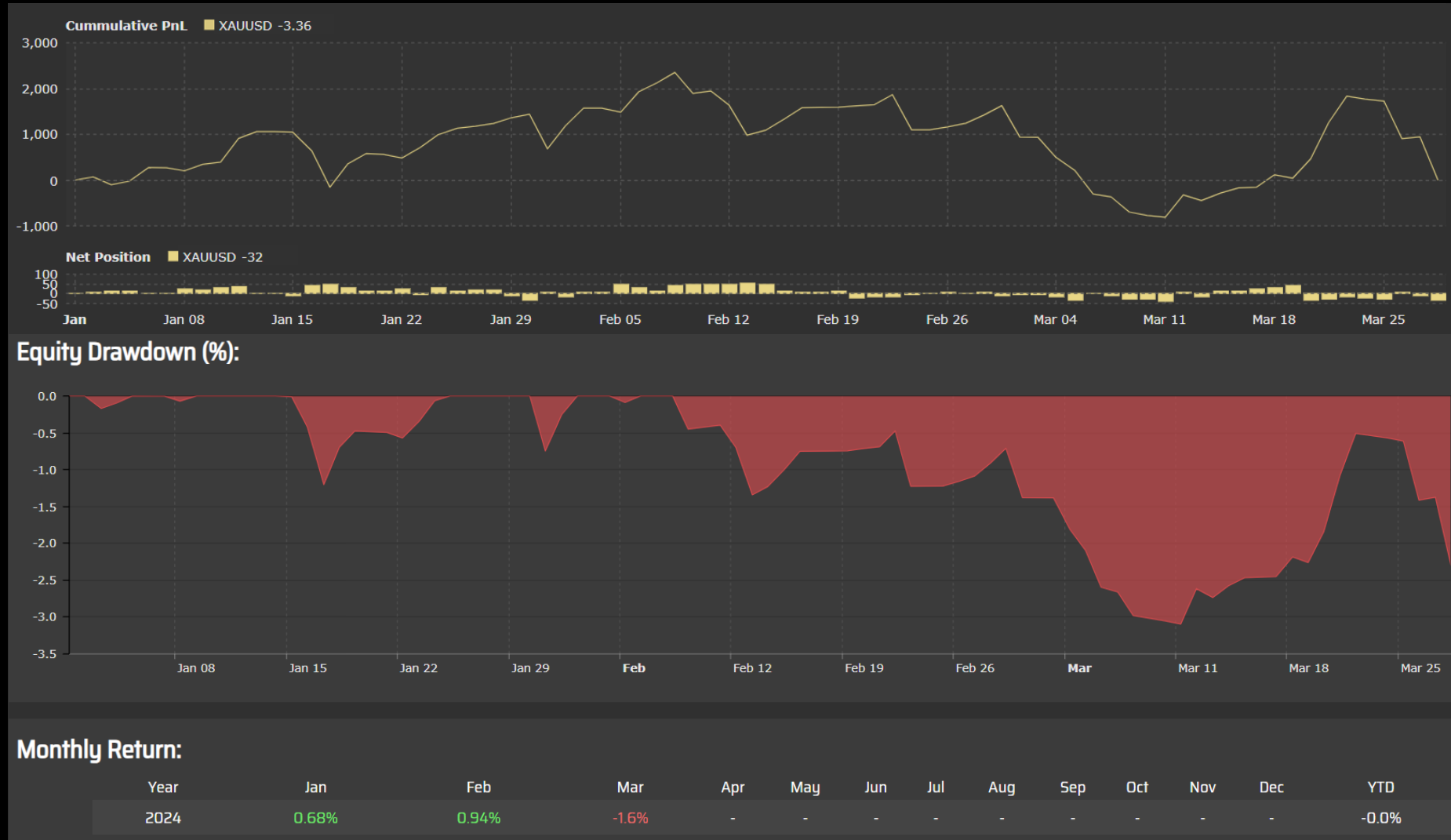
class AlgoEvent:
    def __init__(self):
        self.lasttradetime = datetime(1970,1,1)

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()

    def on_marketdatafeed(self, md, ab):
        if md.timestamp >= self.lasttradetime + timedelta(hours=1):
            self.lasttradetime = md.timestamp
            self.mm_order(md, 1, 'open')
            self.mm_order(md, -1, 'open')

    def mm_order(self, md, buysell, openclose):
        order = AlgoAPIUtil.OrderObject(
            instrument = md.instrument,
            orderRef = 1,
            volume = 0.01,
            openclose = openclose,
            buysell = buysell,
            ordertype = 1, #0=market_order, 1=limit_order, #2=stop_order
            timeinforce = 60*60*24 #cancel unfilled order every day, unit in second
        )
        if buysell==1:
            order.price = md.bidPrice*0.999
        elif buysell==-1:
            order.price = md.askPrice*1.001
        self.evt.sendOrder(order)
```

Market Making Strategy Backtest



Conclusion

- Even rebates can provide stable income, a market maker still need to manage the risks of taking unwanted positions.
- A key point to implement a market making strategy successfully is to maintain the net position as close to zero as possible over time, while maximizing the number of filled pairs.
- Some improvements to above strategy:
 - Only open a set of pair trades when the market is detected to be ranging
 - Early cut loss if the market is detected to be trending in opposite direction
 - Set a wider spread if the market is less liquid; and tighter spread for liquid market
 - Set a wider spread when market is volatile; and tighter spread for less volatile market

Key Takeaways

- Reveals order book insights through real-time market depth and liquidity dynamics.
- Common trading ideas/strategies through order flow analysis
- Market impact of a large order and strategies to mitigate
- The basics of Queuing theory for modelling order execution dynamics and waiting times in an order book
- High frequency trading focuses on rapid execution to capture small price movements.
- Most arbitrage strategies require high frequency trading to capture price discrepancies
- Understand the principles for some popular high frequency strategies
 - Triangular Arbitrage Strategy
 - Market Making Strategy